

В этой главе мы отойдем от рассмотрения архитектур нейронных сетей и коснемся инженерных основ, которые во многом и определяют решения, которые принимаются при проектировании современных моделей. Рассмотрим обучение нейронных сетей на GPU. Но начнем с главного вопроса: а зачем нам вообще GPU?

Линейные слои, свёртки, механизм внимания, нормализации и другие операции в нейронных сетях работают не с одним числом, а с большими тензорами. Линейный слой, например, описывается матричным умножением — матрица объектов $X \in \mathbb{R}^{s \times D}$ (где s — длина последовательности) умножается на матрицу весов $W \in \mathbb{R}^{D \times D}$ и матричным сложением — к каждой строке x прибавляется вектор смещения b .

При перемножении или сложении матриц каждый элемент результирующей матрицы вычисляется независимо от других. При вычислении на CPU, эта информация не дает нам почти никакого преимущества, поскольку операции выполняются последовательно (по сравнению с GPU). Однако, GPU позволяют проводить вычисления параллельно и спроектированы для максимизации пропускной способности (**throughput**) — количества операций за единицу времени.

В свою очередь CPU позволяют проводить вычисления с минимальной задержкой (**latency**), однако будут значительно уступать GPU в пропускной способности.

Таким образом, чем больше вычислений мы можем выполнять параллельно, тем большее преимущество имеет GPU перед CPU. А как мы ранее рассматривали, нейронные сети, в частности трансформеры, очень хорошо распараллеливаются.

Однако не все операции нейронной сети одинаково хорошо ускоряются на GPU. Если операция выполняет мало арифметики, но много читает или пишет в память, то она может упираться не в скорость вычислений, а в скорость чтения и записи памяти. Например, при выполнении поэлементных операций, таких как сложения векторов или матриц.

Проблема усугубляется при появлении редукций: операций, “схлопывающих” массив чисел в одно число. Это могут быть, например, операции сложения, нахождения минимума, среднего или максимума. Например, при вычислении функции Softmax

$$\text{Softmax}_j(x) = \frac{\exp(x^{(j)})}{\sum_{k=1}^D \exp(x^{(k)})}$$

необходимо вычислить сумму $\sum_{k=1}^D \exp(x^{(k)})$ — выполнить редукцию. А функция Softmax необходима для вычисления механизма внимания. Также сложности возникают при вычислении нормализаций, поскольку они требуют подсчёта статистик, в ходе которых вызываются операции редукции — нахождение среднего и суммы.

GPU также плохо справляется с выполнением множества мелких операций. Во-первых, потому что запуск вычислений на GPU происходит на

CPU и требует некоторого времени. Во-вторых, потому что в начале вычисления GPU должен считать данные из памяти, а в конце вычислений — уже записать новые данные в память.

Перед тем, как решать проблемы вычисления нейронных сетей на GPU, необходимо рассмотреть то, как они устроены внутри.

1 Анатомия GPU

1.1 Streaming Multiprocessor

GPU состоит из большого числа вычислительных блоков, которые называются **Streaming Multiprocessors (SM)**. SM является основной единицей исполнения вычислений на GPU. Грубо говоря, GPU можно представить как набор множества SM, каждый из которых умеет независимо выполнять часть общей работы.

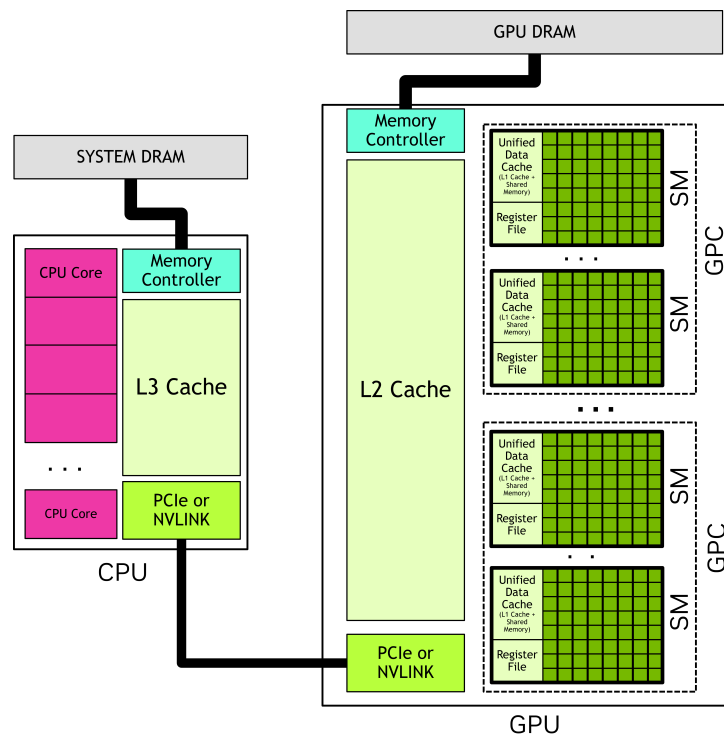


Рис. 1: Архитектура NVIDIA GPU

Внутри SM находятся основные аппаратные компоненты, которые используются при выполнении программ: CUDA-ядра, тензорные ядра, регистры, разделяемая память (shared memory) и варп-планировщики (warp

schedulers).

1.2 Потоки и варпы

Когда мы запускаем вычисления на GPU, задача разбивается на большое число **потоков (threads)**. Каждый поток выполняет одну и ту же программу, но обычно работает со своими данными.

Например, при сложении двух векторов, один поток может отвечать за вычисление одного элемента итогового вектора:

$$c_i = a_i + b_i$$

В таком случае, разные потоки считают разные элементы массива независимо.

GPU, однако, не исполняет такие операции полностью независимо по одному элементу. Они группируются в **варпы (warps)**.

Варп — это группа из 32 потоков, которые исполняются вместе. Все потоки внутри варпа в каждый момент исполняют одну и ту же инструкцию, но могут работать с разными данными. Варп является базовой единицей исполнения инструкций на GPU.

Такой принцип исполнения называется **SIMT** — Single Instruction, Multiple Threads. Одна инструкция применяется сразу к группе потоков. Так, при сложении векторов, все потоки исполняют инструкцию сложения, но каждый поток работает со своим индексом.

Однако, если мы решим выполнить сложение с условием по маске, то задача становится сложнее. Допустим, при значении маски на текущем индексе равном 1, мы выполняем сложение, а при значении маски равном 0 — вычитание. Тогда, если в одном варпе для двух потоков попадутся разные значения маски по их индексу, то варпу придется выполнять 2 разные инструкции — выполнить сложение или вычитание. Такая ситуация называется **расхождением варпа (warp divergence)**.

В этом случае варпу придется разделить выполнение программы на 2 ветки: ту, в которой он выполняет сложение для одних индексов, и ту, в которой он выполняет вычитание для других индексов. Ветки будут вынуждены выполняться последовательно: во время выполнения такой ветки, варп отключает потоки, которые в ней не участвуют. Это плохо, поскольку мы хотим выполнять вычисления параллельно. Поэтому при написании программ на GPU стоит избегать ветвлений, которые могут привести к расхождению варпа.

1.3 CUDA-ядра

CUDA-ядра — это арифметико-логические устройства внутри SM, они выполняют обычные арифметические операции над целыми числами (int) и над числами с плавающей точкой (float), такие как сложение и умножение, а также логические операции, операции сравнения и другие.

Важно, что один поток не соответствует одному CUDA-ядру. Поток — программная единица исполнения, а CUDA-ядро — аппаратное устройство, которое выполняет инструкции. Инструкции варпов уже исполняются аппаратными блоками внутри SM.

1.4 Тензорные ядра

CUDA-ядра хорошо подходят для простых арифметических операций, индексной арифметики, но для выполнения матричных умножений используются специализированные вычислительные блоки внутри SM — **тензорные ядра**.

Тогда как CUDA-ядра выполняют скалярные инструкции, тензорные ядра спроектированы таким образом, чтобы выполнять небольшие операции матричного умножения. Однако, они не предназначены для другой логики.

1.5 Варп-планировщики

Как мы уже обсудили, потоки на GPU не исполняются по одному, а варпами. Однако, сам по себе варп ещё не выполняет вычисления. Внутри SM есть специальные аппаратные блоки, которые выбирают, какой варп будет исполняться в данный момент — **варп-планировщики (warp schedulers)**. Обычно в одном SM четыре варп-планировщика.

В каждый момент времени работы программы на одном SM находится несколько активных варпов — варпов, занявших ресурсы SM. В зависимости от архитектуры, максимальное число активных варпов может быть 32, 48 или 64, однако в случае нехватки ресурсов на SM, это число может быть ниже.

Часть из них готова к выполнению, а часть может ждать данные из памяти или завершения предыдущих инструкций. Задача варп-планировщика — выбрать готовый активный варп и отправить его следующую инструкцию на выполнение.

Если варп запрашивает данные из глобальной памяти, результат может прийти не сразу. CPU в такой ситуации пытается уменьшить задержку за счёт сложных кэшей, предсказаний ветвлений и других инженерных хитростей.

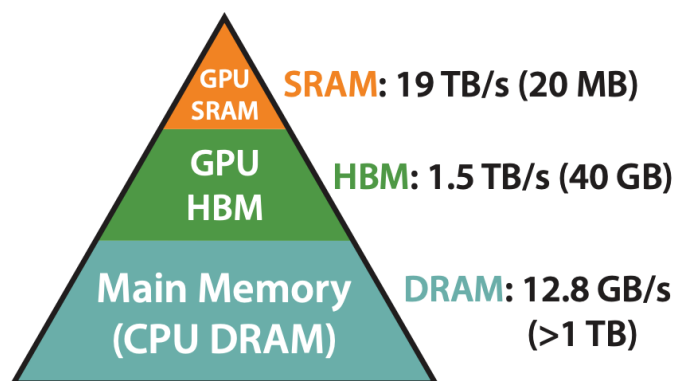
GPU в свою очередь не столько уменьшает задержку, сколько перекрывает ее другой полезной работой: если один варп все еще ждет данные, SM может выполнять инструкции другого варпа.

2 Иерархия памяти

Мы рассмотрели вычислительные блоки GPU. Однако вычисления проводятся над данными, которые где-то необходимо хранить до вычислений и куда-то записывать после.

Хранить данные надо в памяти. Но обращение к памяти не мгновенно и требует некоторого времени. Логично, что те данные, которые чаще всего используются, необходимо держать в наиболее быстрой памяти. При этом, при работе с нейронными сетями часто необходимо хранить гигабайты параметров.

К сожалению, невозможно сделать память одновременно и самой быстрой и при этом максимально объёмной.



**Memory Hierarchy with
Bandwidth & Memory Size**

Рис. 2: Иерархия памяти из статьи
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Таким образом формируется **иерархия памяти**: на одном уровне мы получаем наиболее быструю память, но жертвуем вместимостью, на другом уровне — имеем компромисс, на еще одном, наоборот, получаем очень объёмную память, но жертвуем её скоростью.

2.1 Обмен данными между хостом и девайсом

Наиболее объёмная и дешёвая память — это память диска. На диске мы можем хранить сотни терабайт данных. Однако обращение к такой памяти — самое дорогое. Такие обращения необходимо выполнять асинхронно с остальными операциями, подгружая “наперёд”.

После того, как мы считываем данные с диска, например текст для обучения или веса модели, мы помещаем их в оперативную память CPU.

Обычно, CPU работает со своей памятью, а GPU — со своей. Хотя бывают и исключения, при котором возможно общее физическое адресное пространство. В остальных же случаях, перед тем, как выполнять вычисления

на GPU, данные нужно физически переместить из памяти CPU в память GPU, иначе у GPU просто не будет доступа к этим данным.

CPU также называют **хост (host)**, а GPU — **девайс (device)**.

Копирование данных с хоста на девайс не бесплатное. По возможности, если хватает памяти GPU, все параметры модели загружают на девайс до конца обучения, а данные — до конца итерации.

Если же данные, параметры или активации не помещаются в память девайса, их часть выгружают в память хоста. Такой метод называется **cpu offloading**.

Память CPU управляется операционной системой. Она разбита на **страницы (pages)** — небольшие непрерывные участки виртуальной памяти. Операционная система может перемещать страницы, выгружать их, отображать их на разные физические адреса и управлять ими независимо.

Для обычной CPU программы это удобно, но для копирования с CPU на GPU — нет. Девайс и контроллер передачи данных должны работать с физически доступными участками памяти. Если страница может быть перемещена операционной системой во время копирования, то перед копированием её необходимо разместить в **закрепленном буфере (pinned buffer)**.

Поэтому для эффективного копирования данных с хоста на девайс используется **закреплённая память (pinned memory)**. Мы буквально закрепляем страницу на время копирования и не даём операционной системе её переносить, тем самым уменьшая накладные расходы, которые возникают при копировании данных в закрепленный буфер (pinned buffer) (а также, что крайне важно, позволяем выполнять копирование асинхронно).

2.2 Глобальная память (GMEM)

Основная память GPU называется **глобальной памятью (GMEM)**.

Такая память имеет большой объём и высокую пропускную способность по сравнению с памятью хоста. Однако среди всех остальных уровней иерархии памяти GPU она же — самая медленная и может занимать сотни тактов. Часто задача оптимизации вычислений на девайсе сводится к минимизации количества обращений к глобальной памяти.

Однако записывать данные в GMEM необходимо, поскольку именно так происходит общение между разными программами на одном девайсе. Так, в глобальную память сохраняются активации слоёв во время фазы прямого распространения обучения нейронной сети. Эти активации слоёв потом используются при вычислении обратного распространения. Состояния оптимизатора и градиенты также хранятся в GMEM.

2.3 Разделяемая память (SMEM)

Разделяемая память (shared memory, SMEM) — следующий уровень иерархии. Она намного меньше глобальной памяти, но и доступ к ней значительно быстрее, примерно несколько десятков тактов. SMEM используется

для хранения данных, которые нужны нескольким потокам, исполняющимся на одном SM.

Мы можем загрузить в SMEM данные из GMEM единожды на время работы программы, а переиспользовать их много раз, что особенно особенно понадобится для матричного умножения. Это что-то вроде кэша программы, который разработчик организует самостоятельно.

Важная проблема при использовании разделяемой памяти — **конфликты банков**. SMEM физически разбита на несколько банков. Если несколько потоков одновременно обращаются к адресам, попадающим в один и тот же банк, доступ может выполняться медленнее.

2.4 Память регистров (RMEM)

Самая быстрая память в иерархии — это регистровая память. Регистры находятся внутри SM. Обычно они используются для хранения локальных переменных потока: индексов, промежуточных значений и так далее.

Операции выполняются над данными, находящимися в регистрах. Если SMEM мы можем не использовать, читая из GMEM напрямую, то использовать RMEM мы обязаны.

Регистры очень быстрые, но их ресурсы ограничены и потоки не могут обмениваться друг с другом данными из RMEM. Если регистров не хватает, то компилятор может использовать более медленную память — **local memory**. Такая ситуация называется **register spilling**. Поэтому очень важно следить за регистровым давлением.

2.5 L1 и L2 кэш

Помимо явно управляемой памяти, в GPU есть кэши. Они делятся на два уровня: L1 и L2.

L1 кэш находится внутри SM. Он кэширует часть обращений к глобальной памяти. На современных видеокартах NVIDIA L1 кэш и SMEM часто используют общий физический ресурс. Соотношение размеров L1 кэша и SMEM не фиксировано. Чем больше используется SMEM, тем меньше остается места под L1.

L2 кэш является общим для GPU. Через него происходят обращения разных SM к глобальной памяти.

В отличие от SMEM, L1 и L2 кэши работают автоматически.

2.6 Согласованный доступ к памяти

Для эффективной работы с глобальной памятью важно не только сколько данных мы читаем, но и то, как именно мы их читаем.

Потоки исполняются варпами. Если потоки одного варпа читают соседние адреса памяти, GPU может объединить эти обращения в одну крупную **транзакцию** памяти, что кратно понижает стоимость обращения к

памяти. Это называется **согласованным доступом к памяти (coalesced memory access)**.

То есть, если потоки читают элементы

$$x[i], x[i + 1], x[i + 2], \dots$$

то такой доступ согласованный (коалесцированный). Но если читаются элементы

$$x[i], x[i + D], x[i + 2D], \dots$$

то обращения к памяти будут хуже объединяться в одну транзакцию, вплоть до того, что на каждый такой элемент необходимо будет выделить отдельную транзакцию. Но зачем кому то читать данные таким странным образом?

Тензоры записаны в памяти в виде длинной последовательности элементов. Форма задается с помощью размерностей осей. Для матрицы есть две основных формы записи в памяти — **по строкам (row-major)** и **по колонкам (column-major)**. В первом случае элементы строки матрицы записываются по порядку. Во втором случае, по порядку записываются уже элементы колонки матрицы.

Во время матричного умножения, нам необходимо найти скалярное произведение строки и колонки матрицы. Чтобы доступ к памяти был коалесцированным, необходимо, чтобы левая матрица была записана в row-major форме, а правая — в column-major.

Также часто полезно, чтобы размеры тензоров и смещения в памяти были выровнены и кратны удобным значениям: размеру варпа, ширине шины памяти (векторизованной загрузки). Если не сделать тензоры кратными заранее, в GPU-программе его можно западдить до нужного значения (но это накладные расходы).

3 Метрики производительности GPU

Мы разобрались с устройством вычислений на GPU и иерархией памяти. Но перед тем, как оптимизировать вычисления нейронных сетей, разберемся, как оценивать какие решения выгодно использовать при оптимизации, а какие — нет.

Важно также понимать, чем именно ограничена та или иная операция: скоростью вычислений, скоростью работы с памятью или неудачным паттерном доступа к данным, инициализацией кернелов или синхронизациями.

Для этого используют метрики производительности GPU.

3.1 Задержка и пропускная способность

Первое что приходит в голову, когда сравнить производительность двух программ — давайте оценивать время, которое требуется для выполнения операции или программы.

Эта метрика называется **задержкой (latency)**. Мы можем оценивать как время выполнения всей программы на GPU, так задержку её составляющих: чтения памяти, различных вычислений и записи результата.

Другая метрика, которая часто используется для оценки производительности GPU — это **пропускная способность**. Грубо говоря, это тот объём работы, которая совершается за единицу времени. Например, при обучении или инференса трансформера, мы можем оценивать количество обработанных в секунду токенов.

Нам важно минимизировать задержку и максимизировать пропускную способность.

3.2 FLOP/s

Одна из основных характеристик вычислительной производительности — это количество операций с плавающей точкой, которые GPU способен выполнять за единицу времени, **FLOP/s — floating point operations per second**.

Чтобы оценить FLOP/s, достаточно вычислить количество операций с плавающей точкой FLOPs, необходимых для выполнения программы, и разделить на время работы программы.

$$\text{FLOP/s} = \frac{\text{FLOPs}}{t}$$

Важно отличать пиковое значение FLOP/s от достижимого. В характеристиках GPU обычно указывается пиковое значение — теоретический максимум, который на практике почти недостижим.

Также FLOP/s сильно зависит от типа данных. Один и тот же GPU может иметь разные значения для FP32, FP16, BF16, FP8 или FP4 типов. О них мы поговорим позднее.

3.3 Пропускная способность памяти (memory bandwidth)

Для памяти же одна из основных метрик — это **пропускная способность памяти (memory bandwidth)**. Она показывает, сколько байт данных можно прочесть из памяти или записать в нее за единицу времени.

Если программа читает R байт и записывает W байт за время t , то пропускную способность памяти можно оценить как

$$\text{MemoryBandwidth} = \frac{R + W}{t},$$

также, аналогично FLOP/s, важно различать пиковое значение и достижимое.

Пропускная способность памяти — особенно ценная метрика при выполнении операций, в которых мало арифметики, но много обращений к памяти. Например, при сложении двух векторов

$$c_i = a_i + b_i$$

для каждого элемента необходимо прочитать 8 байт (4 байта для a_i и 4 байта для b_i в FP32 типе) и записать 4 байта. То есть, ради одной операции сложения необходимо передать 12 байт.

Такая функция почти не нагружает вычисления, но нагружает память. Поэтому при оптимизации таких программ важно в первую очередь максимизировать пропускную способность памяти.

3.4 Арифметическая интенсивность

Какие то программы могут быть ограничены пропускной способностью памяти в большей степени, а какие то — в меньшей.

Для оценки, в чем программа ограничена сильнее — в вычислениях или в чтении памяти, используется такая метрика как **арифметическая интенсивность**. Давайте просто разделим FLOP/s на пропускную способность памяти:

$$I = \frac{\text{FLOP/s}}{\text{MemoryBandwidth}}$$

Однако, FLOP/s и MemoryBandwidth неудобно использовать, поскольку обе эти метрики зависят от времени. Поэтому упростим:

$$I = \frac{\frac{\text{FLOPs}}{t}}{\frac{W+R}{t}} = \frac{\text{FLOPs}}{W+R}$$

Теперь арифметическая интенсивность показывает, сколько арифметических операций выполняется на один байт данных, прочитанный или записанный.

3.5 Roofline-модель

Если арифметическая интенсивность низкая, то программа делает мало вычислений на каждый байт данных. Тогда она скорее всего будет ограничена скоростью памяти. Это типично для поэлементных операций (elementwise), редукций, нормализаций, функций активации. Такие операции могут стать бутылочным горлышком в производительности при обучении нейронных сетей.

Если же арифметическая интенсивность высокая, то операция проводит много вычислений над уже загруженными данными. Тогда она может быть ограничена уже скоростью вычислений. Это характерно, например, для больших матричных умножений (но только тех, которые хорошо оптимизированы).

Первый сценарий называется **memory-bound** — программа ограничена скоростью памяти. Второй же — **compute-bound**, когда программа ограничена скоростью вычислений.

Для грубой оценки производительности программы или операции часто используют **roofline-модель**. Производительность операции не может быть выше, чем позволяют мощности девайса (иначе упираемся в compute-bound сценарий), и не может быть выше, чем позволяет пропускная способность памяти (иначе будет memory-bound ограничение).

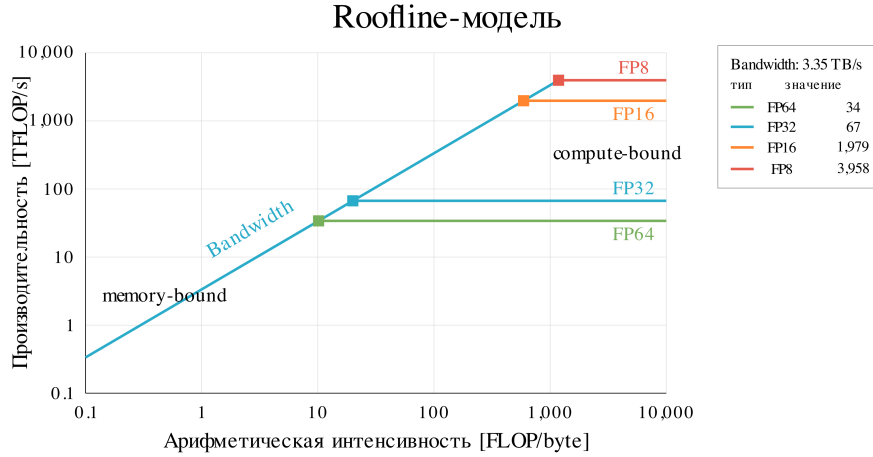


Рис. 3: Roofline-модель.

Максимальная производительность, которую можно получить при ограничении памятью равна:

$$\text{Performance} \leq \text{ArithmeticIntensity} \cdot \text{MemoryBandwidth}$$

В то же время, производительность не может превысить пиковую скорость вычислений PeakFLOP/s:

$$\text{Performance} \leq \text{PeakFLOP/s}$$

Поэтому поэтому максимальную производительность можно записать как:

$$\text{Performance} \leq \min(\text{ArithmeticIntensity} \cdot \text{MemoryBandwidth}, \text{PeakFLOP/s})$$

3.6 Occupancy

Roofline-модель позволяет оценить, в какой области применять оптимизации: в памяти или в вычислениях. Но не дает информации, какие ресурсы GPU используются в текущей программе.

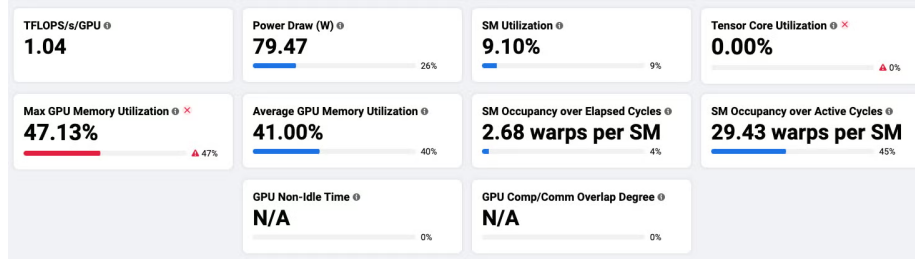


Рис. 4: Метрики производительности GPU в MAIProf.

А ресурсы внутри SM потребляются разные: регистры, тензорные ядра, варп-планировщики, SMEM. И при достижении лимита по одному из эти ресурсов приведет к тому, что остальные будут простаивать.

Например, пусть программа использует слишком много регистров на поток или слишком много SMEM. Тогда на одном SM сможет разместиться меньше потоков и, соответственно, меньше варпов. А при малом количестве варпов на программу планировщику тяжелее снижать задержку через перекрытие времени выполнения варпов. В результате SM будет частично простаивать во время работы программы.

Для описания этого эффекта используется метрика **occupancy**:

$$\text{Occupancy} = \frac{W_{\text{active}}}{W_{\text{max}}},$$

где W_{active} — количество активных варпов во время работы программы, а W_{max} — максимальное количество активных варпов для конкретной архитектуры.

Важно помнить, что occupancy — лишь одна из метрик, по которой можно судить о производительности и по которой можно делать выводы, как оптимизировать программу. Например, программа может иметь высокий occupancy, потому что выполняет матричные вычисления наивным образом, не нагружая SMEM и RMEM. Но такая программа будет memory-bound.